

PROJECT TERMINUS

Snorkel AI — Comprehensive Technical Overview

An Autonomy-First Agent Benchmarking Platform

Platform	Terminus — Terminal AI Benchmarking
Developed By	Stanford University & Laude Institute Researchers
Framework	Harbor AI Evaluation & Benchmarking
Primary Goal	Evaluate & Benchmark Coding Agents in CLI Environments
Community Focus	Expert Engineers & AI Researchers

1. What is Terminus?

Terminus is a specialized, autonomy-first agent platform built specifically for evaluating how Large Language Models (LLMs) operate in terminal and coding environments. It functions as a neutral sandbox testbed where developers and AI researchers can rigorously test the autonomous coding and debugging capabilities of different models.

The platform was created by researchers at Stanford University and the Laude Institute, targeting professionals with deep knowledge of coding and real-world engineering problem solving.

1.1 Core Value Proposition

- Build challenging, real engineering tasks
- Benchmark frontier LLMs against these tasks
- Advance the state of agentic coding
- Reward expert contributors for their domain knowledge

2. Harbor — The Evaluation Framework

Harbor is the AI agent evaluation and benchmarking framework that powers the Terminus platform. It serves a dual role within the Snorkel ecosystem:

2.1 Primary Role

Harbor provides the foundational infrastructure for evaluating coding agents in CLI and containerized environments.

2.2 AI Model Registry

In recent years, Harbor has been adopted as an AI model registry to securely store, version, and share machine learning models across multi-cloud environments. This enables reproducible benchmarking across diverse deployment setups.

3. How Coding AI Agents Work in CLI & Container Environments

CLI AI coding agents (such as Aider, Claude Code, or OpenHands) are automated assistants that live in your terminal. Because they are granted access to the filesystem to write and run code autonomously, running them inside a container environment is essential to prevent them from altering the local machine.

3.1 The Architecture

The Sandbox (MicroVM / Container)

Tools like Docker Sandboxes spawn a lightweight, isolated microVM that contains the agent's execution environment.

Workspace Scoping

Instead of the entire hard drive, the container is given access only to the specific project folder or repository it needs to edit, limiting blast radius.

3.2 The Execution Flow

1. Agent Initialization — The CLI agent is launched inside the container using an execution command (e.g., `devcontainer exec --workspace-folder`).
2. Perception — The agent reads workspace files using environment-supplied APIs.
3. Action — The agent decides how to solve the given prompt (e.g., debugging a Python script or writing a new endpoint) and executes bash commands inside the container.
4. Environment Controls — The agent can spin up secondary databases or testing containers inside its own sandbox without affecting the host machine.
5. Review & Merge — Once the task is complete, the agent's changes are committed to a distinct Git branch, leaving the main codebase untouched until reviewed.

4. How TerminalBench Works

The platform hosts a large variety of tasks designed to test and improve skills across the entire technology spectrum — from quick bug fixes to complex system architecture problems.

4.1 Supported Challenge Categories

Category	Description
Machine Learning	Model training, fine-tuning, evaluation tasks
Data Science	Data pipelines, EDA, statistical analysis
Cyber Security	Vulnerability assessment, CTF-style challenges
Full Stack Web	Frontend + Backend + API integration
System Design	Architecture, scalability, distributed systems
Cloud Infrastructure	Terraform, Kubernetes, cloud provisioning
Algorithms	Data structures, optimization, complexity
Database Optimization	Query tuning, indexing, schema design

4.2 Supported Languages

Contributors can create solutions in the following languages:

Bash • C • C++ • JavaScript • Kotlin • Lua • PHP • Perl • Python • Rego • SQL • Shell • Terraform • TypeScript • Vue

5. Task Skeleton Templates

When creating a new benchmark task, contributors must select one of three skeleton templates:

Template	When to Use
Regular Task Skeleton	All non-UI-Building and non-Milestone tasks
UI Task Skeleton	All UI-Building subtype tasks
Milestone Task Skeleton	Tasks that contain multiple milestones

6. Step-by-Step Task Submission Workflow

6. Download the correct task skeleton template (Regular / UI / Milestone)
7. Extract and rename the folder
8. Write task instructions and configure metadata
9. Set up the Docker environment
10. Create and test your solution locally
11. Write and verify tests

12. Run the Oracle Agent (validates solution passes all test cases)
13. Test with real agents — GPT-4 and Claude Opus 4.6 (run each 2–3 times to gauge pass rate)
14. Run LLM as Judge (LLMaJ) locally
15. Final Verification — ensure all checks pass
16. Create ZIP file and submit to platform (check rubric generation checkbox)
17. Review CI feedback and iterate; review and edit generated rubric for accuracy
18. When CI passes, submit for human reviewer approval

Important: Tasks must have a pass rate of less than 80% against real agents to be accepted and qualify for payment.

7. Agent Types

7.1 Oracle Agent

The Oracle Agent runs the provided solution in a containerized environment (solution/solve.sh). It executes three times within the task environment to verify that all test cases pass. This confirms the task is solvable and the solution is correct.

7.2 NOP Agent (Dummy Agent)

The NOP Agent is a dummy agent used for:

- CI/CD validation
- Benchmark sanity checks
- Evaluation pipeline debugging
- Detecting faulty task definitions

If the NOP Agent passes all tasks, it signals that the task definition is invalid — a properly constructed task should not be solvable by a no-operation agent.

8. Key Runtime Tools

8.1 Tmux — The Agent Runtime

Tmux (Terminal Multiplexer) has become the standard backend environment for parallel AI agents in the Terminus ecosystem.

Feature	Description
---------	-------------

Persistence	Agents execute long-running builds in detached sessions. If the main server disconnects, agents continue uninterrupted.
Parallel Orchestration	A single machine can split into multiple independent panes and windows; one orchestration agent can delegate to many sub-agents.
Programmatic Control	MCP servers interact directly with tmux via scripts or sockets to automate complex CLI workflows.

8.2 Asciiinema — The Observational Layer

Asciiinema is a lightweight, text-based terminal session recorder used as the observability layer for Terminus agents.

Feature	Description
Observability & Debugging	Records all terminal inputs and outputs as precise text-based data (not heavy video files), enabling exact replay of agent actions.
Replayability & Review	Developers can replay agent actions in a browser or terminal exactly as they occurred, and copy-paste command outputs.
Session Documentation	Recordings can be embedded in documentation or shared in pull requests to showcase agent reasoning and execution steps.

8.3 Combined Usage — Tmux + Asciiinema

In advanced setups, developers run multiple AI agents inside dedicated tmux sessions and use asciiinema to record and debug the agents' exact sequences. Tools like tvmux combine the two, generating dedicated asciiinema recordings as the agent navigates between tmux panes.

9. Rubric — Scoring Criteria

The Rubric defines the professional scoring criteria used to evaluate autonomous AI software engineering agents. It is auto-generated during submission and must be reviewed and edited by the task author for accuracy and completeness before final reviewer approval.

A well-constructed rubric ensures fair, reproducible, and objective assessment of each agent's performance on the benchmark task.

10. Final Verification Checklist

Before submitting a task for review, confirm all of the following:

- Oracle Agent passes all test cases (3 runs)
- All LLMaJ (LLM as Judge) checks pass
- Tested against real agents — GPT-4 and Claude Opus — with pass rate below 80%
- All required files are present and correctly structured
- CI feedback is clean and all pipeline stages pass
- Rubric has been reviewed and edited for accuracy and completeness
- ZIP file has been created with the rubric generation checkbox selected